

Making vulnerability prediction more practical: Prediction, categorization, and localization

Chongyang Liu^a, Xiang Chen^{b,*}, Xiangwei Li^a, Yinxing Xue^{a,*}

^a School of Computer Science and Technology, University of Science and Technology of China, Hefei, China

^b School of Information Science and Technology, Nantong University, Nantong, China

ARTICLE INFO

Keywords:

Deep learning
Pre-trained language model
Semantic analysis
Vulnerability detection

ABSTRACT

Context: Due to the prevalence of software vulnerabilities, vulnerability detection becomes a fundamental problem in system security.

Objective: To solve this problem, academics and industries have made great efforts to propose deep-learning-based (DL-based) approaches but these attempts have three main limitations: (1) perform poorly on real-world projects (e.g., Accuracy below 74.33% and F1 below 73.55%); (2) perform poorly in catching vulnerable patterns due to incomplete code representations; (3) mostly perform coarse-grained function-level prediction and lack interpretability analysis.

Methods: In this paper, we propose VulPCL, a BLSTM and CodeBERT based approach, which makes the first attempt to perform vulnerability prediction, categorization, and localization automatically within a framework. To alleviate the above-mentioned limitations, our VulPCL considers multi-dimension (i.e., text-based, sequence-based, and graph-based) representations to catch latent vulnerable patterns and multi-model training to learn high-level semantics.

Results: Through experiments on four real-world datasets containing 114+ CWE (Common Weakness Enumeration) types spanning from 2005 to 2022, we find that our VulPCL outperforms the baselines by (1) 13.51%~60.64% and 14.34%~180.23% on Accuracy, and F1 respectively on vulnerability prediction; (2) 10.32%~46.79%, and 10.71%~127.80% on Accuracy, and macro-F1 respectively on vulnerability categorization; (3) 9.23%~36.54% on Top-10 Accuracy on vulnerability localization.

Conclusion: These results indicate that our VulPCL is considerably more accurate, effective, fine-grained, and practical than previous studies. Besides, our further analyses show that VulPCL is indeed capable of capturing all vulnerability lines, and the result of line-level vulnerability localization is consistent with the function-level vulnerability prediction as the increase of predicted lines. Thus making VulPCL more interpretable than previous studies. Our additional investigation also shows that VulPCL effectively detects the Most Dangerous 25 CWEs in 2022, which is instructive for security researchers.

1. Introduction

Software vulnerabilities are mistakes in the design, development, or configuration of software that could be exploited by attackers to perform malicious behaviors [1]. Unresolved vulnerabilities may cause potential threats to the software and computer systems. In particular, criminal groups could take software vulnerabilities to steal the private data of personnel or companies, which results in severe economic losses. With the rapid increase in the number of vulnerabilities that were publicly reported through the Common Vulnerabilities and

Exposure (CVE) [2] recently, vulnerability detection has become a fundamental problem in system security.

Since the traditional vulnerability prediction methods rely on rich human expertise and intense manual labor, researchers have made great efforts to propose various DL-based vulnerability prediction approaches [3–19] to help security analysts find the vulnerabilities fastly. Due to the differences between abstract semantic information learned by deep learning algorithms and understood by human experts, researchers adopted many types of code representations to promote the alleviation of the semantic gap. For instance, text-based approaches

* Corresponding authors.

E-mail addresses: lcyyy@mail.ustc.edu.cn (C. Liu), xchencs@ntu.edu.cn (X. Chen), lixw20@mail.ustc.edu.cn (X. Li), yxxue@ustc.edu.cn (Y. Xue).

use the plain text of source code [3,11,20]; slice-based approaches use the slices induced by data dependency (DD) and control dependency (CD) [7,8,14–16]; graph-based approaches [4–6,9,10,18,19] use the property graphs such as AST and CPG generated from the source code of functions.

Although previous studies have made certain achievements, they still suffer from limitations in learning comprehensive semantics for complex real-world programs and performing fine-grained vulnerability prediction. First, previous approaches perform poorly on real-world projects, indicating their dependencies on datasets and their impracticality. Second, the low Recall and F1 score of previous studies imply their poor effectiveness in capturing vulnerability patterns. Third, DL-based works as a black box model and the underlying mechanism is poorly explained [21,22], leading to confusion about how a DL-based model makes such a prediction and whether it can catch all the vulnerable lines in a function. Thus lacking interpretability analysis makes the coarse-grained DL-based function-level vulnerability prediction controversial.

In this study, we propose VULPCL, a BLSTM [23] and CodeBERT [24] based method that can perform vulnerability prediction, categorization, and localization automatically within a framework. To alleviate the above-mentioned limitations, we first consider multi-dimension representations (i.e., text-based, sequence-based, and graph-based representations) to extract semantics at different levels. Specifically, we use the plain text of source code (PTSC), flattened CFG and DFG sequence (FCDS), and completed PDG/AST graph (CPAG) to capture latent vulnerability patterns, while using multi-model training for better semantic learning. Second, we perform vulnerability categorization to further investigate the effectiveness of VULPCL in vulnerability characteristics caching and semantics learning. Third, we perform more fine-grained vulnerability localization and further analyze: (1) the ability of VULPCL on capturing vulnerable lines; (2) the variety of prediction accuracy as the increase of predicted lines.

Through an extensive evaluation of our VULPCL on FFmpeg, QEMU, Linux Kernel (the self-collected dataset that contains the most vulnerability types), and Big-Vul datasets with over 114+ different CWE types spanning from 2005 to 2022, we find that VULPCL: (1) outperforms the state-of-the-art baseline approaches (i.e., SySeVR [8], Devign [9], and REVEAL [10]) by 13.51%~60.64% and 14.34%~180.23% on Accuracy, and F1 respectively on vulnerability prediction; (2) outperforms the state-of-the-art baseline approaches (i.e., CodeBERT [24], CNN [25], and GRU [26]) by 10.32%~46.79%, and 10.71%~127.80% on Accuracy, and macro-F1 respectively on vulnerability categorization; (3) outperforms the state-of-the-art baseline approaches (i.e., LINE-VUL [20], LIG [27], Saliency [28], DeepLift [29,30], DeepLiftSHAP [31], GradientSHAP [31] and CppCheck [32]) by 9.23%~36.54% on Top-10 Accuracy on vulnerability localization.

In summary, we make the following contributions:

- **Fine-grained vulnerability detection with interpretability.** We propose VULPCL, a fine-grained vulnerability detection approach based on RNN and Transformer. Besides vulnerability prediction and categorization, VULPCL further analyzes the vulnerability patterns and identifies vulnerable lines in code. Through an extensive evaluation, VULPCL outperforms the state-of-the-art baselines on vulnerability prediction, categorization and localization, which is promising that VULPCL can address various limitations of existing approaches.
- **Exploring multi-dimension representation learning for vulnerability detection.** We consider the PTSC, FCDS, and CPAG, leveraging CodeBERT and BLSTM to extract multi-dimensional features from these representations, which are then applied in vulnerability detection. Through an ablation study, we find that the design of each dimension of representation is effective. Besides, the combination of representations from all dimensions achieves the best performance.
- **Dataset.** To better evaluate the generalization of our VULPCL, besides FFmpeg, QEMU, and Big-Vul datasets, we collect and manually

labeled a dataset from an open-source project Linux Kernel with 17,478 C/C++ functions and 89 CWE types spanning from 2005 to 2022.

To support the open science community, we make our dataset and scripts of VULPCL publicly available at: <https://github.com/liucyy/VULPCL>.

2. Background and challenges

In this section, we introduce preliminary knowledge, which includes code representations, existing datasets, vulnerability types involved in the experiment, and the challenges to previous studies.

2.1. Code representation

Program semantics learning is the foundation for DL-based vulnerability detection tasks. To better preserve and learn semantic information in C/C++ open-source projects, researchers transform the plain text of project files into various code representations, i.e. Abstract Syntax Tree (AST), Control Flow Graph (CFG), Data Flow Graph (DFG), Program Dependency Graph (PDG), and Code Property Graph (CPG).

2.2. Existing datasets for function-level vulnerability prediction

To evaluate the effectiveness of proposed DL-based vulnerability detection models, researchers need to collect datasets with labels vulnerable or non-vulnerable. Depending on how the code examples are gathered, existing datasets can be categorized as follows.

Synthetic datasets. The code examples in these datasets are artificially synthesized with known vulnerability patterns (e.g., SARD [33]), thus the vulnerabilities are easy to be identified.

Semi-synthetic datasets. In these datasets, code examples are derived artificially and vulnerabilities are isolated from the original context (e.g., examples of NVD [34]), causing them not capable of fully imitating the complexities of real-world vulnerable patterns.

Real-world datasets. The code examples in these datasets are collected from open-source projects such as FFmpeg, QEMU, Linux Kernel, and chrome (e.g., FFmpeg dataset [9]). Due to requirements (such as quality, scalability, maintainability, and security), code examples in real-world datasets are more complicated than those in the above two types of datasets.

2.3. Vulnerability types

The Common Weakness Enumeration (CWE) community [35] executes the vulnerability classification and assigns an id to each of them. Corresponding to each CWE id, we can learn the relevant vulnerability information such as name, description, and examples. Our experiments involve more than 114 types of vulnerabilities including *Improper Input Validation* (i.e., CWE-20), *Use After Free* (i.e., CWE-416), *Buffer Overflow* (i.e., CWE-120, CWE-121, and CWE-122), *Resource Management Error* (i.e., CWE-399), *Cross-site Scripting* (i.e., CWE-79), *SQL Injection* (i.e., CWE-89), and *Code Injection* (i.e., CWE-94).

2.4. Challenges to previous studies

Previous DL-based vulnerability detection approaches are often evaluated at the function level. These approaches combined NLP and program analysis techniques to improve the prediction accuracy and achieved promising performance on their considered datasets. However, there are three main challenges of these studies, which will be illustrated as follows.

Poor performance on real-world projects. On the one hand, some approaches (e.g., VulDeePecker [7], SySeVR [8]) can achieve high accuracy and low FPR on synthetic and semi-synthetic datasets (e.g., NVD [34] and SARD [33] datasets) due to the known patterns and context independence of vulnerabilities, the accuracy decreases

<pre>1 void main(){ 2 int MAX = 6; 3 char a[MAX], b[MAX]; 4 char str = "asdsdsff"; 5 bool flag = true; 6 for(int i=0; i < 8; i++){ 7 a[i] = i+1; 8 if(i > 5) 9 free(a); 10 if(flag == true) 11 flag = false; 12 strcpy(b, str); 13 } 14 }</pre>	<table><tr><th>State-of-the-art work</th><th>Representation</th><th colspan="2">Captured Vulnerability trace</th></tr><tr><td>Ground Truth</td><td>None</td><td>8, 9, 6, 7</td><td>5, 10, 12</td></tr><tr><td>Russell et al.</td><td>Plain text</td><td>6, 7, 8, 9</td><td></td></tr><tr><td>VulDeePecker</td><td>Slice with DD</td><td>2, 3, 7, 9</td><td>2, 3, 12</td></tr><tr><td>SySeVR</td><td>Slice with CD and DD</td><td>6, 7, 8, 9</td><td>6, 10, 12</td></tr><tr><td>Devign</td><td>CPG</td><td>8, 9, 6, 7</td><td>5, 10, 11, 12</td></tr><tr><td>REVEAL</td><td>CPG</td><td>8, 9, 6, 7</td><td>5, 10, 11, 12</td></tr><tr><td>VulPCL(ours)</td><td>Multi-dimension representation</td><td>8, 9, 6, 7</td><td>5, 10, 12</td></tr></table>	State-of-the-art work	Representation	Captured Vulnerability trace		Ground Truth	None	8, 9, 6, 7	5, 10, 12	Russell et al.	Plain text	6, 7, 8, 9		VulDeePecker	Slice with DD	2, 3, 7, 9	2, 3, 12	SySeVR	Slice with CD and DD	6, 7, 8, 9	6, 10, 12	Devign	CPG	8, 9, 6, 7	5, 10, 11, 12	REVEAL	CPG	8, 9, 6, 7	5, 10, 11, 12	VulPCL(ours)	Multi-dimension representation	8, 9, 6, 7	5, 10, 12
State-of-the-art work	Representation	Captured Vulnerability trace																															
Ground Truth	None	8, 9, 6, 7	5, 10, 12																														
Russell et al.	Plain text	6, 7, 8, 9																															
VulDeePecker	Slice with DD	2, 3, 7, 9	2, 3, 12																														
SySeVR	Slice with CD and DD	6, 7, 8, 9	6, 10, 12																														
Devign	CPG	8, 9, 6, 7	5, 10, 11, 12																														
REVEAL	CPG	8, 9, 6, 7	5, 10, 11, 12																														
VulPCL(ours)	Multi-dimension representation	8, 9, 6, 7	5, 10, 12																														

Fig. 1. A motivation example of the ability of the six DL-based vulnerability detection tools (including our proposed approach) to capture the vulnerable traces, the code snippet contains vulnerabilities of Use After Free and Buffer Overflow.

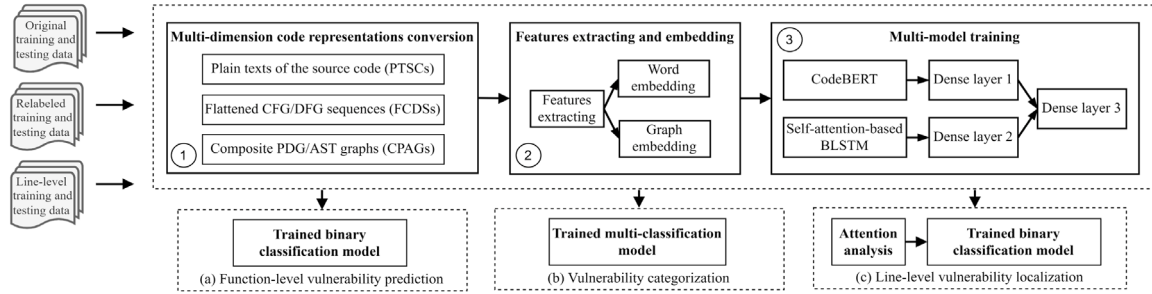


Fig. 2. A overview of the proposed vulnerability prediction, categorization and localization framework (VulPCL).

by almost more than 50% on real-world datasets [10] (such as FFmpeg and QEMU datasets [9]). On the other hand, some approaches (e.g., LineVul [36]) can perform well on extremely imbalanced real-world datasets (e.g., Big-Vul [37]), the accuracy and F1 score decrease by more than 36% on balanced real-world datasets (as shown in Section 5 and VI). Besides, almost all the proposed DL-based approaches performed badly in the real scenario with low accuracy, recall, and F1 score (shown in Section 5). Therefore, a more effective DL-based framework should be designed to make it more practical for real-world projects.

Poor effectiveness in capturing vulnerable patterns. To find the cause of the poor performances of the state-of-the-art works [7–11] on real-world projects, we analyze their ability to capture the vulnerability traces (ordered vulnerable lines derived from representations used by the works). Fig. 1 shows a motivation example. The vulnerable function *main* with two types of vulnerabilities (i.e., *use after free* and *buffer overflow*) is shown on the left of Fig. 1. According to the representations used by the works, we list the dangerous traces that can be caught by each work and the ground truth in the right of Fig. 1. The result points out that these baselines cannot or only partly identify vulnerability patterns in function *main*.

Poor interpretable coarse-grained prediction. The previous studies [7–11,15] can only predict whether a function is vulnerable or not but cannot categorize and localize the vulnerability within a model. Questions are often asked such as how the DL-based models make such predictions, and whether they can capture all the vulnerable lines in a function, causing the coarse-grained vulnerability prediction is poorly interpretable. In particular, the source code of functions contains many lines, it is difficult to localize the vulnerable ones.

To address the above-mentioned challenges, we consider multi-dimension code representations and combine the self-attention-based BLSTM and CodeBERT model to perform fine-grained, interpretable, and practical DL-based vulnerability detection.

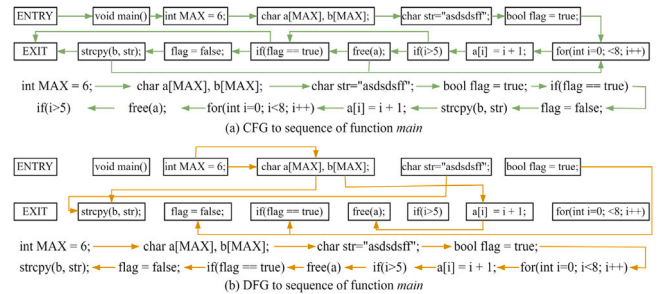


Fig. 3. A example of the transformation of CFG and DFG to corresponding FCDSS.

3. Approach

3.1. Overview

In this section, we propose VulPCL, a BLSTM and CodeBERT based approach (shown in Fig. 2) that can perform vulnerability prediction, categorization, and localization automatically within a framework. We consider multi-dimension representations to catch latent vulnerable patterns and multi-model training for better semantic learning. On the function-level prediction, we extract PTSCs, FCDSS, and CPAGs from the source code of functions, feed the word/graph embedding vectors of them into CodeBERT and BLSTM models, then train the model and predict whether a function is vulnerable or not. On the vulnerability categorization, we relabel the vulnerable functions and model it as a multi-classification task. We process the source code of vulnerable functions as described above, and output the categories of each function. On vulnerability localization, we define it as line-level vulnerability prediction and perform attention analysis for each line in functions based on function-level prediction, the output is the vulnerable lines. The function-level vulnerability prediction and categorization are designed for the first two challenges, while the line-level localization is designed for the last challenge.

3.2. Data processing and model training

Step①: Multi-dimension code representations conversion. Fig. 1 shows that the state-of-the-art slice-based and CPG-based approaches are poorly effective in vulnerability pattern capturing. To address this challenge, we consider text-based representation (i.e., PTSC), sequence-based representation (i.e., FCDS), and graph-based representation (i.e., CPAG), which is helpful to catch all the vulnerabilities in function *main*. In this step, all the property graphs are constructed by Joern [38], a tool for analyzing and generating property graphs of code. First, we get the original sequences from the source code of functions and these sequences serve as PTSCs. Then, we use Joern to automatically flatten the CFGs and DFGs, and the generated code sequences are saved in SVG files. We directly derive the FCDSs from these SVG files. To provide a more intuitive illustration, we take the function *main* shown in Fig. 1 as an example and show the conversion result in Fig. 3.

Algorithm 1: Completing PDG with AST

Input: Vertex set of PDG – V^{PDG}
 Edge set of PDG – E^{PDG}
 Vertex set of AST – V^{AST}
 Edge set of AST – E^{AST}
Output: Vertex set of CPAG – V^{CPAG}
 Edge set of CPAG – E^{CPAG}

```

1 for  $v_j^p \in V^{PDG}, v_i^p \in V^{PDG}$  do
2    $V^{CPAG} \leftarrow V^{CPAG} \cup v_j^p$ ;
3    $V^{CPAG} \leftarrow V^{CPAG} \cup v_i^p$ ;
4   if  $\exists e_{ji}^p \in E^{PDG}$  then
5      $E^{CPAG} \leftarrow E^{CPAG} \cup e_{ji}^p$ ;
6   end
7 end
8 for  $v_r^a \in V^{AST}, v_q^a \in V^{AST}$  do
9   if  $\exists e_{rq}^a \in E^{AST}$  then
10     $E^{CPAG} \leftarrow E^{CPAG} \cup e_{rq}^a$ ;
11    if  $v_r^a \in V^{PDG}$  then
12       $V^{CPAG} \leftarrow V^{CPAG} \cup v_q^a$ ;
13    end
14    if  $v_q^a \in V^{PDG}$  then
15       $V^{CPAG} \leftarrow V^{CPAG} \cup v_r^a$ ;
16    end
17  end
18 end
```

Finally, we design an algorithm to complete the PDGs with corresponding ASTs to get the CPAGs. As shown in Algorithm 1, we first initialize the vertex set of the corresponding CPAG with all the vertexes in PDG (Line 1 to 3), if there exists an edge between two vertexes, then the edge is merged into the edge set of the CPAG (Line 5). For each edge in the corresponding AST, if one of its vertexes belongs to the vertex set of CPAG, the edge is merged into the output edge set (Line 9 to 10) and another vertex is merged into the output vertex set (Line 11 to 15).

Step②: Features extraction and embedding. Since we use three types of representations to facilitate the retention of syntax and semantic information, we take different feature extraction and embedding methods for different representations.

For the PTSCs, CodeBERT uses Byte-Pair-Encoding (BPE) [39,40] to tokenize a source code of function into sub-tokens. Let $s = \{t_1, \dots, t_n\}$ denotes the sub-token sequence with the length of n . Before inputting s into Transformer blocks, it should be padded with a special classification token $[CLS]$ and an ending token $[EOS]$ at the beginning and ending respectively. Thus the input is embedded as $s_{embed} = e_{[CLS]}, e_{t_1}, \dots, e_{t_n}, e_{[EOS]}$.

For the FCDSs, we convert them into token sequences using white space characters (e.g., white space and newline), then embed the generated token sequences by the word2vec [41] model, which has been widely used in text mining tasks [42].

For the CPAGs, we utilize the information of AST and CFG nodes, extract the code in nodes, and the node types as features. Note that, each code in the nodes is regarded as a single token. We embed the extracted features by the graph embedding tool Deepwalk [43] to capture: (1) the topology of CPAGs; (2) the information of vertices and the relationships between different vertexes; (3) the relationships relevant to CPAGs and their subgraphs.

Step③: Multi-model training. For the embedding vectors derived from multi-dimension representations, we employ two models to perform multi-model training. The vectors corresponding to the PTSCs are trained by CodeBERT, while the vectors corresponding to the FCDSs and CPAGs are trained by a self-attention-based BLSTM network. Let $W^{PTSC} = \{w_1^{PTSC}, \dots, w_m^{PTSC}\}$, $W^{FCDS} = \{w_1^{FCDS}, \dots, w_n^{FCDS}\}$, and $W^{CPAG} = \{w_1^{CPAG}, \dots, w_k^{CPAG}\}$ denote the three types of trained vectors, after reshaping them through *Dense layer 1* and *Dense layer 2* (shown in Fig. 2), we concatenate them as follows:

$$W = \{w_1^{PTSC}, \dots, w_m^{PTSC}, w_1^{FCDS}, \dots, w_n^{FCDS}, w_1^{CPAG}, \dots, w_k^{CPAG}\} \quad (1)$$

Then the vector W will be input into different kinds of *Dense layer 3* (shown in Fig. 2) to perform different downstream tasks.

3.3. Tasks definition

3.3.1. Vulnerability prediction

The function-level vulnerability prediction is designed to evaluate the effectiveness of the proposed multi-dimension representations and the multi-model training technique. We defined it as a binary classification task, thus a function is labeled as “0” for non-vulnerable and “1” for vulnerable. After data processing and model training, a function is predicted as vulnerable or not.

3.3.2. Vulnerability categorization

To explore whether our VulPCL is capable of categorizing the vulnerabilities, we relabel the vulnerable functions by considering top-10 vulnerabilities and model it as a multi-classification task. If there are two or more types of vulnerabilities in a function, we assign it multiple labels. In this task, the number of classes in *Dense layer 3* is set to 11 (the 11th is “others”). Through Step① to ③, the model then outputs the category of each function.

3.3.3. Vulnerability localization

We conduct a more fine-grained vulnerability localization to address the last challenge in Section 2. Based on the function-level prediction, we define vulnerability localization as a line-level vulnerability prediction task. Since the core component of the CodeBERT model is the multi-head attention mechanism, the tokens with high attention scores have a higher contribution to prediction results, thus the lines with high attention scores are more likely to be vulnerable. After data processing and model training, we calculate the attention score of each line in functions by accumulating the attention score of each token in the line, and consider the lines with higher scores as the vulnerable ones. Due to the previous studies lacking the interpretability analysis, in this task, we further analyze: (1) the ability of VulPCL on capturing vulnerable lines; (2) the variety of prediction accuracy as the number of the predicted line goes from 1 to the maximum line number of functions.

4. Experimental setup

In this section, we introduce our research questions, studied datasets, baselines, evaluation metrics, and experiment setting.

Table 1

The statistic of the datasets used in our experiments.

Dataset	Functions	Vulnerable functions	Non-Vulnerable functions	Graphs	Vulnerable graphs	Non-Vulnerable graphs
FFmpeg [9]	9769	4981	4788	9472	4821	4651
QEMU [9]	17 549	7479	10 070	17 231	7342	9889
Big-Vul [37]	188 636	10 900	177 736	19 976	6040	13 936
Linux Kernel	17 478	4093	13 385	16 995	3902	13 093

Table 2

The CWE ID, vulnerability description, and the number of functions for each of the top-10 vulnerabilities in Linux Kernel dataset.

CWE ID	Vulnerability description	Vul-funcs count
CWE-20	Improper Input Validation	483
CWE-416	Use After Free	303
CWE-362	Concurrent Execution using Shared Resource with Improper Synchronization ('Race Condition')	276
CWE-200	Exposure of Sensitive Information to an Unauthorized Actor	245
NVD-CWE-Other	other	233
CWE-119	Improper Restriction of Operations within the Bounds of a Memory Buffer	228
CWE-264	Permissions, Privileges, and Access Controls	221
CWE-476	NULL Pointer Dereference	186
NVD-CWE-noinfo	Insufficient Information	185
CWE-400	Uncontrolled Resource Consumption	178

4.1. Research questions

To comprehensively evaluate our proposed VulPCL on different granularity, we design the following research questions.

RQ1: How does VulPCL perform on vulnerability prediction compared with state-of-the-art baselines?

RQ2: How does VulPCL perform on vulnerability categorization compared with state-of-the-art baselines?

RQ3: How does VulPCL perform on vulnerability localization compared with state-of-the-art baselines?

4.2. Studied datasets

Gathering and using high-quality datasets is significant for software vulnerability prediction, and we experiment on four different datasets to address the challenges in Section 2. The statistical information of these datasets is shown in Table 1. In our function-level experiment, we use the FFmpeg and QEMU datasets provide by Zhou et al. [9] while the other two datasets in [9] are not available.

The FFmpeg and QEMU datasets are based on C/C++ language, each containing 9769 and 17,549 functions while most vulnerabilities in functions are memory-related such as buffer overflow, memory leak, and crash. However, there are the following limitations in the datasets: (1) the vulnerability types involved are insufficient; (2) the ground truths of specific vulnerability types are lacking; (3) not applicable to vulnerability categorization. To address these limitations and perform vulnerability categorization, we collected a more comprehensive dataset from a large C-language open-source project Linux Kernel.

The collection and labeling of our Linux Kernel dataset. Since the *diff* information in vulnerability-related patch files of source code contain the specific fixed vulnerable lines, we retrieved the vulnerable comments in the GitHub Advisory Database [44] and crawled the vulnerability-related patches and corresponding code files through the web links. Note that each of the collected C files contains two or more functions, thus we split each one into function-level (i.e., each file contains a single function only) by using the tool developed by Wang et al. [45]. According to the *diff* information, we manually labeled the functions as vulnerable (i.e., the functions which are modified) and

non-vulnerable (i.e., the functions which are not modified) from the source code of the versions previous to the revision. The first and third authors who are experienced with dataset labeling spent 225 human hours manually labeling each function in the Linux Kernel dataset. To ensure the quality of data labeling, we involve another expert to manually inspect the labeling.

Our collected dataset contains vulnerabilities submitted from 2006 to 2022, with a total of 89 different types of vulnerabilities (such as *Use After Free*, *Buffer Overflow*, and *Resource Management Errors*) and 17,478 functions. Table 2 shows the CWE ID, vulnerability description, and the number of vulnerable functions (vul-funcs count) for each of the top 10 vulnerabilities.

On line-level vulnerability prediction, we experiment on the subset of Big-Vul dataset with the line-level ground truths (i.e., which lines in functions are vulnerable) provided by Fan et al. [37]. In Big-Vul, there are 6040 vulnerable functions with line-level labels and 94.3% of the functions in the dataset are non-vulnerable, it is extremely imbalanced. To this reason, we select the 6040 vulnerable functions and sample 13,936 non-vulnerable functions to keep the class distribution consistent with the other three datasets.

4.3. Baselines

To our best knowledge, we have not found any study that can predict whether a function is vulnerable or not, categorize the vulnerabilities, and localize the vulnerable lines subsequently, we make a comprehensive comparison between VulPCL and three types of baselines respectively.

For the vulnerability prediction task, we consider three state-of-the-art baselines.

DeSign [9] uses the Code Property Graphs (CPGs) with natural code sequences, applying the GGNN model and a *Conv* layer which is consist of 1-D convolution and dense neural networks.

SySeVR [8] is a bidirectional-GRU-based model, adopting slices of Library/API Function Call, Array Usage, Pointer Usage, and Arithmetic Expression induced by data dependency and control dependency.

REVEAL [10] is a GGNN-based pipeline that utilizes the CPGs as the code representations, with *Triplet Loss* as the loss function.

For the vulnerability categorization task, we not found any related DL-based study, thus we consider the general CodeBERT [24], CNN [25], and GRU [26] networks as the baselines.

For the localization of the vulnerable lines, we consider the following seven state-of-the-art baselines.

LINEVUL [20] is a CodeBERT-based model with BPE method.

Model-Agnostic Techniques contain Layer Integrated Gradient (LIG) [27], Saliency [28], DeepLift [29,30], DeepLiftSHAP [31], GradientSHAP [31] and CppCheck [32].

4.4. Evaluation metrics

In the classification task, there are four statistical indicators: True Positive and True Negative denote the number of functions that are correctly classified as vulnerable ones or non-vulnerable ones, False Positive (FP) and False Negative (FN) denote the number of functions that are misclassified as vulnerable ones or non-vulnerable ones. Based on the four indicators, we adopt ten metrics to evaluate the effectiveness of VulPCL, i.e. Accuracy, Precision, Recall, F1 (F1-score), macro-Precision, macro-Recall, macro-F1, micro-Precision, micro-Recall, and micro-F1.

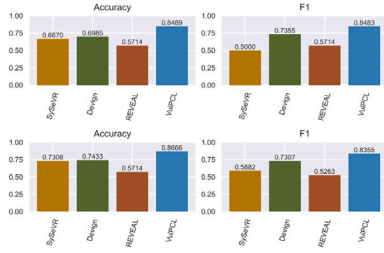


Fig. 4. The function-level performance of VulPCL on the FFmpeg and QEMU datasets compared to the state-of-the-art baselines.

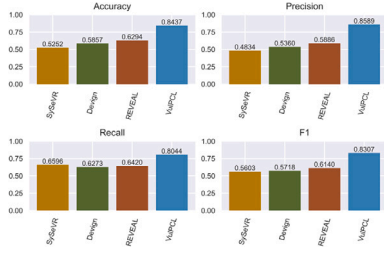


Fig. 5. The function-level performance of VulPCL on the FFmpeg+QEMU dataset compared to the state-of-the-art baselines.

4.5. Empirical setting

Our VulPCL is executed in Python with the help of the Pytorch framework, BLSTM and CodeBERT model. The CodeBERT model is used to encode and train the PTSCs, each function is embedded as a 768-dimensional numerical vector. We use word2vec to embed each FCDS as a 1024-dimensional numerical vector. Besides, the DeepWalk model is utilized to embed each CPAG as a 256-dimensional numerical vector. The shape of concatenated vector in *Dense Layer 3* is [16, 256*3]. In our framework, we employ the *codebert-base* and a 2-layers BLSTM with a dropout of 0.5 and a self-attention layer. During the training phase, the parameters are optimized by Adam, the batch size is set to 8, the loss function is *ReLU* and the epoch is set to 20.

For each dataset, we split the dataset into training data, validation data, and testing data, which proportion is 80%, 10%, and 10%. For each split, we keep its data distribution consistent with the original dataset. Importantly, each experiment we performed is repeated 5 times and we use the average of five experimental results as the final result. For RQ2, the configurations and execution of baselines are consistent with VulPCL; for RQ1 and RQ3, we use original configurations and executions as clarified in previous studies.

5. Experiment result

5.1. RQ1: How does VulPCL perform on vulnerability prediction compared with state-of-the-art baselines?

Setting. To answer this RQ, we aim to investigate the effectiveness of the proposed multi-dimension representations (i.e., PTSC, FCDS, and CPAG) and the multi-model training technique, and the practicality of VulPCL on function-level vulnerability prediction. We consider three state-of-the-art approaches as baselines, including SySeVR [8], Design [9], and REVEAL [10]. To evaluate the validity and practicality of VulPCL and the baselines, we perform our experiment on FFmpeg, QEMU, Linux Kernel, and the hybrid FFmpeg+QEMU datasets. In this RQ, we utilize the widely used A, P, R, and F1 as the metrics.

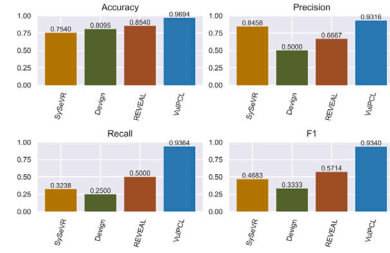


Fig. 6. The function-level performance of VulPCL on Linux Kernel dataset compared to the state-of-the-art baselines.

Result. Fig. 4 presents the experiment results of our VulPCL and the three baselines on the FFmpeg and QEMU datasets. Figs. 5 and 6 present the experiment results of our VulPCL and the three baselines on FFmpeg+QEMU and the Linux Kernel datasets.

On the FFmpeg and QEMU datasets, our VulPCL achieves the best Accuracy of 86.66% and the best F1 of 84.83%. On the FFmpeg+QEMU and Linux Kernel datasets, our VulPCL achieves the best Accuracy of 96.94%, the best Precision of 93.16%, the best Recall of 93.64%, and the best F1 of 93.40%. We find that VulPCL outperforms the baselines by 13.51%~60.64%, 10.14%~86.32%, 21.95%~274.56%, and 14.34%~180.23% in terms of Accuracy, Precision, Recall, and F1, respectively. As shown by the results, our VulPCL achieves the best performances on all the datasets, proving the generalizability of our approach. Different from the state-of-the-art baselines, our VulPCL gets high Recall (above 80.44%) and Precision (above 85.89%), demonstrating that our approach keeps a low FN and FR. Besides, compared to the baselines, our VulPCL achieves the highest Accuracy and F1. We believe the improvement achieved by VulPCL can be attributed to the proposed multi-dimension representations and multi-model training, the former is capable of capturing the potential vulnerability patterns while the latter is beneficial to better learning the high-level semantics.

Answer to RQ1: On function-level vulnerability prediction, our VulPCL outperforms the state-of-the-art baselines by above 10% on all the metrics, proving the effectiveness of (1) proposed multi-dimension representations on vulnerability patterns catching; (2) utilized multi-model training on semantics learning.

5.2. RQ2: How does VulPCL perform on vulnerability categorization compared with state-of-the-art baselines?

Setting. To answer this RQ, we aim to investigate the effectiveness of VulPCL on vulnerability categorization. Since we have not found any related DL-based vulnerability categorization study, we consider the general CodeBERT [24], CNN [25], and GRU [26] networks as the baselines. For a fair comparison, we use the PTSCs, FCDSs, and CPAGs as the input of the CNN and GRU. We perform our experiment on the self-collected Linux Kernel dataset. In Linux Kernel, the functions with two or more vulnerabilities are assigned to multiple labels. For a better evaluation of our VulPCL, we summarize the top-10 vulnerabilities in the Linux Kernel dataset and model the vulnerability categorization as a multi-classification task. In this scenario, we use seven performance measures (i.e., Accuracy, micro-Precision, micro-Recall, micro-F1, macro-Precision, macro-Recall, and macro-F1) to comprehensively evaluate our VulPCL and compare VulPCL with three baselines. We further consider another three metrics (i.e., Precision, Recall, and F1) to investigate the performance of VulPCL in identifying each of the 10 classes of vulnerabilities.

Result. Fig. 7 shows the performance of our VulPCL compared with three baselines. Table 3 shows the performance of VulPCL on each of the 10 classes of vulnerabilities.

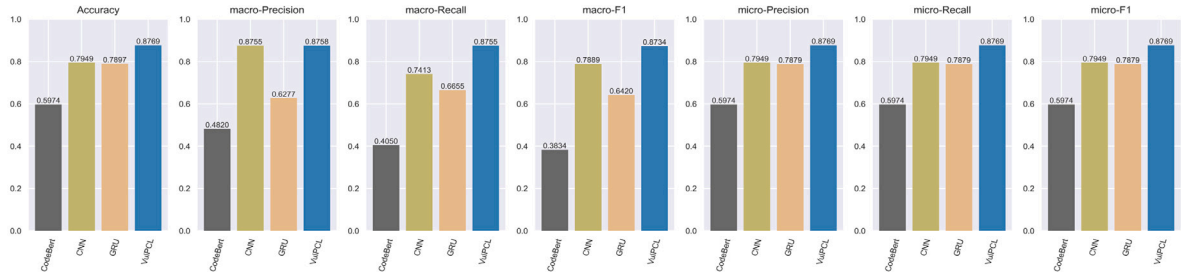


Fig. 7. The vulnerability categorization performance of VulPCL on Linux Kernel compared to three baselines.

Table 3

The Precision, Recall, and F1 of VulPCL on each vulnerability on Linux Kernel dataset.

CWE ID	Precision	Recall	F1
CWE-20	86.27%	89.80%	88.00%
CWE-416	81.82%	84.38%	83.08%
CWE-362	95.83%	92.00%	93.88%
CWE-200	89.66%	89.66%	89.66%
NVD-CWE-Other	82.86%	80.56%	81.69%
CWE-119	77.78%	80.77%	79.25%
CWE-264	100%	83.33%	90.91%
CWE-476	76.92%	100%	86.89%
NVD-CWE-noinfo	95.00%	90.48%	92.68%
CWE-400	87.50%	82.35%	84.85%
others	89.76%	89.76%	89.76%

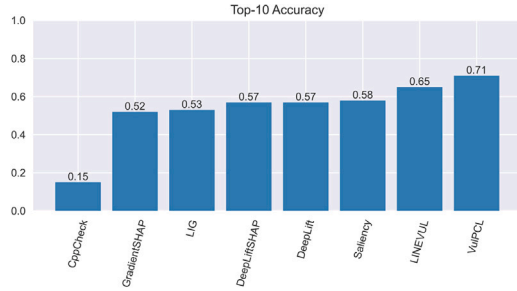


Fig. 8. The line-level performance of VulPCL on Big-Vul compared to seven state-of-the-art baselines on $Accuracy_{top10}$ (Top-10 Accuracy).

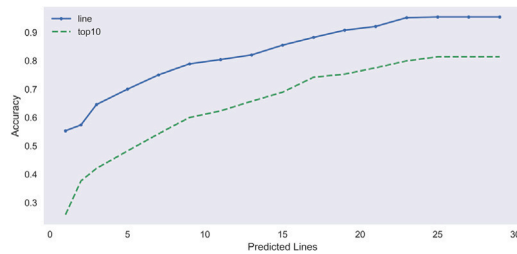


Fig. 9. The variety of $Accuracy_{top10}$ and $Accuracy_{line}$ (Accuracy) as the number of predicted lines goes from 1 to N_{line} (output lines), which generated by VulPCL.

Compared with three state-of-the-art baselines, our VulPCL achieves the best performance on all the seven metrics. We find that VulPCL outperforms the baselines by 10.32%~46.79%, 18.10%~116.17%, 10.71%~127.80%, 10.32%~46.79%, 10.32%~46.79%, 10.32%~46.79% in terms of Accuracy, macro-Recall, macro-F1, micro-Precision, micro-Recall, and micro-F1, respectively. Besides, the Precision, Recall, and F1 of VulPCL on each of the 10 vulnerabilities are above 76.92%, 80.77%, and 79.25%. As shown in Table 2 and Fig. 7, our VulPCL achieves the highest Accuracy, Recall, and F1, demonstrating that our approach is effective on vulnerability categorization without incurring high FN and FP. Table 3 shows that our VulPCL are capable of categorizing each of

the 10 vulnerabilities with high Precision, Recall, and F1. The results are consistent with our motivation, i.e. the proposed multi-dimension representations are beneficial to capture the characteristics of each vulnerability precisely. The good performances of the baselines (such as CNN and GRU) on Accuracy, Recall, and F1 can also further prove the validity of our considered three types of representations (i.e., PTSC, FCDS, and CPAG).

Answer to RQ2: On vulnerability categorization, our VulPCL outperforms three state-of-the-art baselines with high Accuracy, Precision, Recall, and F1. The results are consistent with our motivation, i.e. the proposed multi-dimension representations are beneficial to capture the characteristics of each vulnerability precisely, thus making VulPCL more practical.

5.3. RQ3: How does VulPCL perform on vulnerability localization compared with state-of-the-art baselines?

Setting. To answer this RQ, we aim to investigate the effectiveness of our VulPCL on line-level vulnerability prediction and further perform some interpretability analyses. In this task, a function is correctly predicted if a non-vulnerable function is predicted as non-vulnerable or at least one line in a vulnerable function is identified. For a better evaluation of our VulPCL, let $Accuracy_{top10}$ denote the proportion of correctly predicted vulnerable functions in the top-10 vulnerabilities, $Accuracy_{line}$ denotes the proportion of correctly predicted functions.

We compare our VulPCL to seven state-of-the-art baselines (i.e., LINEVUL [20], LIG [27], Saliency [28], DeepLift [29,30], DeepLiftSHAP [31], GradientSHAP [31], and CppCheck [32]) on $Accuracy_{top10}$. Besides, let N_{line} denotes the maximum number of lines in all functions, we further analyze: (1) whether VulPCL is capable of catching all vulnerable lines in vulnerable functions; (2) the variety of $Accuracy_{line}$ and $Accuracy_{top10}$ as the number of predicted lines increases from 1 to N_{line} .

Result. Fig. 8 shows the performance of VulPCL and the state-of-the-art baselines on the Big-Vul dataset. Fig. 9 and Table 4 show the variety of $Accuracy_{line}$ and $Accuracy_{top10}$, and an example of captured vulnerable lines.

Our VulPCL achieves a $Accuracy_{top10}$ of 71%, which outperforms the baselines by 9.23%~36.53%. As the number of predicted lines increases from 1 to N_{line} , the $Accuracy_{line}$ raises from 55.34% to 95.48%, while the $Accuracy_{top10}$ raises from 25.79% to 81.41%. The result in Fig. 8 shows that besides outperforming the baselines on function-level vulnerability prediction and vulnerability categorization, our VulPCL also outperforms the state-of-the-art approaches to vulnerability localization. Table 4 shows that VulPCL is capable of capturing all the vulnerability lines in a vulnerable function, this directly proves the validity of our proposed multi-dimension representations. Besides, in Fig. 9, as the number of the predicted line increases from 1 to N_{line} , the accuracy improves from line level to function level. The additional analyses imply that the effectiveness of VulPCL is attributed to the proposed multi-dimension representations and the learned high-level

Table 4

An example of how our VULPCL is capable of capturing all the vulnerable lines in a vulnerable function.

Function name	CWE ID	Actual vulnerable Lines	Captured vulnerable Lines
drivers/net/usb/rtl8150.c	CWE-119	2,3,4	2,3,4
virt/kvm/iommu.c	CWE-264	10	10
magick/cache.c	CWE-399	1,2,3,4,5,6,7,8,9,10,11	8,3,4,5,2,7,9,10,6,11,1
ipc/sem.c	CWE-189	2,3	3,2
src/alloc.c	CWE-190	2,3	2,3
src/hashtable.c	CWE-310	1,2,3	2,3,1
third_party/libpng/pngrutil.c	CWE-119	2,3,4,5	2,4,3,5
src/cjson.c	CWE-119	1,2,3,4,5,6,7,8,9,10,11,12	2,8,10,9,6,3,5,7,4,11,12,1
src/cjson.c	CWE-119	1,2,3,4,5,6	2,4,5,3,1,6
fs/ext4/xattr.c	CWE-19	1,2,3	2,3,1

Table 5

Results of the ablation experiment on FFmpeg, QEMU, and Linux Kernel datasets.

Method	FFmpeg				QEMU				Linux Kernel			
	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1
CodeBERT(Source code)	62.39%	65.31%	57.35%	61.07%	68.79%	65.48%	55.42%	60.03%	85.59%	73.35%	59.39%	65.54%
BLSTM(CFG+DFG)	69.94%	67.65%	76.89%	71.98%	80.63%	78.92%	73.94%	76.35%	91.71%	87.86%	84.52%	86.16%
BLSTM(AST+PDG)	75.11%	73.90%	77.94%	75.87%	83.76%	86.27%	73.25%	79.23%	93.47%	90.54%	80.20%	85.06%
VULPCL	84.49%	85.27%	84.39%	84.83%	86.66%	87.29%	80.11%	83.55%	96.94%	93.16%	93.64%	93.40%

Table 6

The Accuracy of VULPCL for the Top-25 Most Dangerous CWEs in Linux Kernel dataset.

Rank	CWE ID	Vulnerability description	TPR	Proportion
1	CWE-787	Out-of-bounds Write	91.67%	11/12
3	CWE-125	Out-of-bounds Read	57.14%	4/7
4	CWE-20	Improper Input Validation	83.05%	49/59
7	CWE-416	Use After Free	92.86%	26/28
12	CWE-190	Integer Overflow or Wraparound	100%	6/6
14	CWE-287	Improper Authentication	100%	1/1
15	CWE-476	NULL Pointer Dereference	93.75%	15/16
17	CWE-119	Improper Restriction of Operations within the Bounds of a Memory Buffer	92.31%	24/26
19	CWE-276	Incorrect Default Permissions	66.67%	2/3
20	CWE-200	Exposure of Sensitive Information to an Unauthorized Actor	100%	20/20
	Total		88.76%	158/178

semantics instead of the datasets themselves. Thus making VULPCL more interpretable and practical than previous studies.

Answer to RQ3: On vulnerability localization, our VULPCL outperforms the baselines by above 9.23%. Further analyses make it promising that the effectiveness of VULPCL is attributed to the proposed multi-dimension representations and the learned high-level semantics. Thus making VULPCL more interpretable and practical than previous studies.

6. Discussions

In this section, we further analyze and discuss the results of VULPCL, and provide some recommendations for future works. Following this, we explore the threat to the validity of our VULPCL.

6.1. Why VULPCL performs well on function-level vulnerability prediction?

We perform an ablation study to explore the contribution of the considered three representations of our VULPCL on function-level vulnerability prediction. In general, our approach utilizes PTSCs, FCDs, and CPAGs to learn the potential semantics from the code of the functions. To better understand each of the three representations, we conduct the following three control methods:

CodeBERT (Source Code) inputs the PTSCs of functions directly into the pre-trained CodeBERT model, then classify by a dense layer.

BLSTM (CFG+DFG) uses FCDs as the input, while the parameters are consistent with VULPCL.

BLSTM (AST+PDG) uses CPAGs as the input with the same parameters as VULPCL.

Table 5 shows the result of the ablation study. Even though the CodeBERT model achieves the highest performances in many tasks such as code search and code generation [46,47], it is the worst on function-level vulnerability prediction. This indicates the features extracted from the property graphs generated by program analysis tools are more effective than those extracted directly from the code of functions. We find that the method using CPAGs performs better than the one using FCDs, which may be attributed to the following reasons: (1) the CPAGs contain more semantic information than the FCDs do; (2) the graph embedding outperforms word embedding. Furthermore, the fusion of the PTSCs, FCDs, and CPAGs achieves the best performances on each dataset, which implies that the proposed composite representation is more beneficial to preserve more semantics.

In our approach, we use the FCDs instead of the composite CFG/DFG graphs to extract semantic features. This can facilitate the efficiency of our VULPCL, but may miss some structural and semantic information. Meanwhile, DeepWalk only considers the similarity of adjacent nodes while ignoring the weights of the edges. Thus, future researchers should: (1) convert the property graphs to sequences with more effective algorithms; (2) use more comprehensive graph embedding algorithms such as SDNE [48] and code2seq [49]; (3) train the extracted graph features with graph neural networks.

6.2. How accurate is VULPCL for identifying the TOP-25 most dangerous CWEs?

Although our VULPCL outperforms three baselines (i.e., CodeBERT, CNN, and GRU) in identifying the top 10 vulnerabilities on the Linux Kernel dataset, there are only four vulnerabilities appear in the top 25 ranking of the 2022 CWE Top 25 Most Dangerous Software Weaknesses [50] (the most extensive and influential issues in the last two years). These weaknesses are dangerous due to they are easy to be found and exploited to attack the computer system. To make VULPCL more instructive to security researchers, we further investigate the accuracy of VULPCL in the top 25 ranking CWEs. For this purpose, we summarize the vulnerabilities that belong to the top 25 rankings

and perform a corresponding experiment. Table 6 shows the result and information related to the vulnerabilities.

On average, our VulPCL achieves a TPR of 88.76% in the most dangerous 25 CWEs in Linux Kernel dataset. The result indicates our VulPCL is also capable of identifying the weaknesses that appear in the top 25 ranking in the Linux Kernel dataset, which implies the effectiveness of VulPCL. However, there are only 10 types of vulnerabilities in the Linux Kernel dataset that appear in the Top 25 ranking while some of them are few in number. *Thus, future researchers should verify our VulPCL on the datasets with more types of vulnerabilities and collect enough code examples for each type.*

6.3. Threats to validity

Threats to the internal validity are primarily potential errors within the execution of our VulPCL framework, which are related to the hyperparameter settings (e.g., dimension of embedding vector, number of hidden layers, and learning rate) we used in our experiments. Previous studies on defect prediction [35,51] imply that the evaluation results may be influenced by different hyperparameter settings. However, it can be too expensive to find the optimal hyperparameters while training CodeBERT and BLSTM simultaneously. To ensure fairness, we utilize the original source code and reproduced code (if the original code is not available) of the state-of-the-art baselines, keeping the hyperparameter settings consistent with the original model. Besides, we utilize the performances reported in the papers as comparisons.

Threats to external validity mainly lie in the projects we experiment on. We use three public datasets and one self-collected dataset, which are gathered from high-popularity open-source C/C++ projects (e.g., FFmpeg, QEMU, Linux Kernel, and Chrome). The datasets contain 114+ CWE types spanning from 2005 to 2022. Although our VulPCL outperforms the state-of-the-art baselines, it is still unknown whether VulPCL does necessarily generalize to other datasets with different CWE types. Thus future researchers should explore the generalizability of VulPCL on other datasets.

7. Related work

In this section, we discuss the novelty and scope of our approach with respect to the reviewed studies of DL-based vulnerability detection and Transformer-based vulnerability categorization.

DL-based vulnerability detection. Researchers proposed various DL-based methods to implement function-level vulnerability prediction tasks. For example, Russell et al. [11] utilized CNN and GRU to extract features which then were trained by the random forest model. Li et al. [7] proposed VulDeePecker, a BLSTM architecture, training on slices of Library/API function calls induced by data dependency. Li et al. [8] further proposed SySeVR to experiment on four kinds of slices (i.e., library/API function call, array usage, pointer usage, and arithmetic expression) induced by control dependency and data dependency. Zhou et al. [9] proposed a GGNN-based model called Devign, which complemented the CPGs with corresponding ASTs to capture the high-level semantic information. Chakraborty et al. [10] proposed ReVeal, which is based on GGNN [36], adopting CPG as the code representation and executing the experiment on real-world datasets. Li et al. [18] proposed IVDetect, which leverages Feature-attention Graph Convolution Network to perform function-level vulnerability and uses GNNExplainer [52] to locate vulnerable sub-graphs in PDGs of code snippets.

Different from existing DL-based function-level approaches, inspired by multi-dimension representation learning in vulnerability fix detection tasks [53–55], VulPCL considers three types of representations (i.e., PTSC, FCDs, and CPAG) to preserve more semantics and learn latent vulnerability patterns, while using appropriate models for each representation. The excellent performances of VulPCL on real-world datasets indicate that VulPCL is more practical than previous studies.

Transformer-based vulnerability detection. Recently, Transformer-based models have achieved promising performances across various SE tasks including vulnerability detection. Thapa et al. [56] conducted an empirical study on the performances of Transformer-based models on function-level vulnerability detection, which specific to two types of vulnerabilities (i.e., Buffer Error and Resource Management Error). Hin et al. [19] utilized CodeBERT and graph attention network [57] to learn function-level and statement-level features of code snippets, respectively. They proposed LineVD to perform statement-level vulnerability detection. Fu et al. [20] proposed LINEVUL, which is a CodeBERT architecture, locating the vulnerable lines with the Multi-head Self-Attention mechanism.

Compared with existing Transformer-based approaches, our VulPCL integrates features derived from PTSCs, FCDs, and CPAGs to capture high-level semantics in code snippets. Besides, VulPCL is capable of exposing the vulnerability of a given code snippet as well as the corresponding CWE type and vulnerable lines.

8. Conclusion

In this paper, we propose VulPCL, a framework based on self-attention-based BLSTM and CodeBERT. Our approach is the first to predict whether a function is vulnerable or not, categorize the vulnerabilities, and localize the vulnerable lines automatically within a framework. To address the challenges in proposed DL-based approaches, VulPCL considers multi-dimension representations to capture the latent vulnerability patterns and adopts a multi-model training method to learn high-level program semantics. We also perform interpretability analysis on line-level vulnerability localization, making VulPCL more practical and interpretable. Compared to the state-of-the-art baselines, our VulPCL is promising: (1) 13.51%~60.64% more accurate on function-level prediction with an improvement of 14.34%~180.23% in F1; (2) 10.32~46.79% more accurate on vulnerability categorization; (3) 9.23%~36.54% more accurate on vulnerability localization. Importantly, our approach achieves an accuracy of 88.76% for the Top-25 Most Dangerous CWEs in 2022, which is instructive for security researchers. Furthermore, we discuss the research challenges that future researchers should explore (e.g., using advanced graph embedding algorithms and GNN networks).

CRedit authorship contribution statement

Chongyang Liu: Data curation, Software, Writing – original draft. **Xiang Chen:** Methodology, Supervision, Writing – review & editing. **Xiangwei Li:** Data curation, Software, Validation. **Yinxing Xue:** Conceptualization, Methodology, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

Xiang Chen and Yinxing Xue have contributed equally to this work and they are co-corresponding authors. This work is supported in part by the Anhui Provincial Department of Science and Technology under Grant 202103a05020009, in part by the National Natural Science Foundation of China under Grant 61972373, in part by the Basic Research Program of Jiangsu Province under Grant BK20201192.

References

- [1] S.M. Ghaffarian, H.R. Shahriari, Software vulnerability analysis and discovery using machine-learning and data-mining techniques: A survey, *ACM Comput. Surv.* 50 (4) (2017) 1–36.
- [2] Common Vulnerabilities and Exposure, [Online] <https://cve.mitre.org/>.
- [3] H.K. Dam, T. Tran, T. Pham, S.W. Ng, J. Grundy, A. Ghose, Automatic feature learning for predicting vulnerable software components, *IEEE Trans. Softw. Eng.* 47 (1) (2018) 67–85.
- [4] G. Lin, J. Zhang, W. Luo, L. Pan, Y. Xiang, POSTER: Vulnerability discovery with function representation learning from unlabeled projects, in: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 2539–2541.
- [5] A.V. Phan, M. Le Nguyen, L.T. Bui, Convolutional neural networks over control flow graphs for software defect prediction, in: *2017 IEEE 29th International Conference on Tools with Artificial Intelligence, ICTAI, IEEE*, 2017, pp. 45–52.
- [6] S. Wang, T. Liu, L. Tan, Automatically learning semantic features for defect prediction, in: *2016 IEEE/ACM 38th International Conference on Software Engineering, ICSE, IEEE*, 2016, pp. 297–308.
- [7] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, Vuldeepecker: A deep learning-based system for vulnerability detection, 2018, arXiv preprint [arXiv:1801.01681](https://arxiv.org/abs/1801.01681).
- [8] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, Sysevr: A framework for using deep learning to detect software vulnerabilities, *IEEE Trans. Dependable Secure Comput.* (2021).
- [9] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, *Adv. Neural Inf. Process. Syst.* 32 (2019).
- [10] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: Are we there yet, *IEEE Trans. Softw. Eng.* (2021).
- [11] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, M. McConley, Automated vulnerability detection in source code using deep representation learning, in: *2018 17th IEEE International Conference on Machine Learning and Applications, ICMLA, IEEE*, 2018, pp. 757–762.
- [12] G. Lin, J. Zhang, W. Luo, L. Pan, O. De Vel, P. Montague, Y. Xiang, Software vulnerability discovery via learning multi-domain knowledge bases, *IEEE Trans. Dependable Secure Comput.* 18 (5) (2019) 2469–2485.
- [13] H.K. Dam, T. Tran, T. Pham, S.W. Ng, J. Grundy, A. Ghose, Automatic feature learning for predicting vulnerable software components, *IEEE Trans. Softw. Eng.* 47 (1) (2018) 67–85.
- [14] X. Cheng, H. Wang, J. Hua, G. Xu, Y. Sui, Deepwukong: Statically detecting software vulnerabilities using deep graph neural network, *ACM Trans. Softw. Eng. Methodol. (TOSEM)* 30 (3) (2021) 1–33.
- [15] Z. Li, D. Zou, S. Xu, Z. Chen, Y. Zhu, H. Jin, Vuldelocator: a deep learning-based fine-grained vulnerability detector, *IEEE Trans. Dependable Secure Comput.* (2021).
- [16] D. Zou, S. Wang, S. Xu, Z. Li, H. Jin, μ VulDeePecker: A deep learning-based system for multiclass vulnerability detection, *IEEE Trans. Dependable Secure Comput.* 18 (5) (2019) 2224–2236.
- [17] G. Lin, S. Wen, Q.-L. Han, J. Zhang, Y. Xiang, Software vulnerability detection using deep neural networks: a survey, *Proc. IEEE* 108 (10) (2020) 1825–1848.
- [18] Y. Li, S. Wang, T.N. Nguyen, Vulnerability detection with fine-grained interpretations, in: *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 292–303.
- [19] D. Hin, A. Kan, H. Chen, M.A. Babar, Linevd: Statement-level vulnerability detection using graph neural networks, in: *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 596–607.
- [20] M. Fu, C. Tantithamthavorn, LineVul: A transformer-based line-level vulnerability prediction, 2022.
- [21] F.-L. Fan, J. Xiong, M. Li, G. Wang, On interpretability of artificial neural networks: A survey, *IEEE Trans. Radiat. Plasma Med. Sci.* 5 (6) (2021) 741–760.
- [22] Y. Zhang, P. Tiño, A. Leonardis, K. Tang, A survey on neural network interpretability, *IEEE Trans. Emerg. Top. Comput. Intell.* (2021).
- [23] A. Graves, J. Schmidhuber, Framewise phoneme classification with bidirectional LSTM and other neural network architectures, *Neural Netw.* 18 (5–6) (2005) 602–610.
- [24] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, et al., Codebert: A pre-trained model for programming and natural languages, 2020, arXiv preprint [arXiv:2002.08155](https://arxiv.org/abs/2002.08155).
- [25] S. Lawrence, C.L. Giles, A.C. Tsoi, A.D. Back, Face recognition: A convolutional neural-network approach, *IEEE Trans. Neural Netw.* 8 (1) (1997) 98–113.
- [26] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio, Learning phrase representations using RNN encoder-decoder for statistical machine translation, 2014, arXiv preprint [arXiv:1406.1078](https://arxiv.org/abs/1406.1078).
- [27] M. Sundararajan, A. Taly, Q. Yan, Axiomatic attribution for deep networks, in: *International Conference on Machine Learning, PMLR*, 2017, pp. 3319–3328.
- [28] K. Simonyan, A. Vedaldi, A. Zisserman, Deep inside convolutional networks: Visualising image classification models and saliency maps, 2013, arXiv preprint [arXiv:1312.6034](https://arxiv.org/abs/1312.6034).
- [29] M. Ancona, E. Ceolini, C. Öztireli, M. Gross, Towards better understanding of gradient-based attribution methods for deep neural networks, 2017, arXiv preprint [arXiv:1711.06104](https://arxiv.org/abs/1711.06104).
- [30] A. Shrikumar, P. Greenside, A. Kundaje, Learning important features through propagating activation differences, in: *International Conference on Machine Learning, PMLR*, 2017, pp. 3145–3153.
- [31] S.M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [32] CppCheck, [Online], <https://cppcheck.sourceforge.io/>.
- [33] N. I. of Standards and Technology, Software assurance reference dataset, 2018, [Online] <https://samate.nist.gov/SRD/index.php>.
- [34] H. Booth, D. Rike, G.A. Witte, et al., The national vulnerability database (nvd): Overview, 2013.
- [35] Common weakness enumeration, 2006, [Online], <https://cwe.mitre.org/index.html>.
- [36] Y. Li, D. Tarlow, M. Brockschmidt, R. Zemel, Gated graph sequence neural networks, 2015, arXiv preprint [arXiv:1511.05493](https://arxiv.org/abs/1511.05493).
- [37] J. Fan, Y. Li, S. Wang, T.N. Nguyen, AC/C++ code vulnerability dataset with code changes and CVE summaries, in: *Proceedings of the 17th International Conference on Mining Software Repositories*, 2020, pp. 508–512.
- [38] F. Yamaguchi, N. Golde, D. Arp, K. Rieck, Modeling and discovering vulnerabilities with code property graphs, in: *2014 IEEE Symposium on Security and Privacy, IEEE*, 2014, pp. 590–604.
- [39] P. Gage, A new algorithm for data compression, *C Users J.* 12 (2) (1994) 23–38.
- [40] R. Sennrich, B. Haddow, A. Birch, Neural machine translation of rare words with subword units, 2015, arXiv preprint [arXiv:1508.07909](https://arxiv.org/abs/1508.07909).
- [41] Word2vec, 2018, [Online], <http://radimrehurek.com/gensim/models/word2vec.html>.
- [42] L. Wolf, Y. Hanani, K. Bar, N. Dershowitz, Joint word2vec networks for bilingual semantic representations, *Int. J. Comput. Linguist. Appl.* 5 (1) (2014) 27–42.
- [43] B. Perozzi, R. Al-Rfou, S. Skiena, Deepwalk: Online learning of social representations, in: *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2014, pp. 701–710.
- [44] GitHub Advisory Database, [Online], <https://github.com/advisories>.
- [45] P. Wang, J. Svajlenko, Y. Wu, Y. Xu, C.K. Roy, CCaligner: a token based large-gap clone detector, in: *Proceedings of the 40th International Conference on Software Engineering*, 2018, pp. 1066–1077.
- [46] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, M. Brockschmidt, Codesearchnet challenge: Evaluating the state of semantic code search, 2019, arXiv preprint [arXiv:1909.09436](https://arxiv.org/abs/1909.09436).
- [47] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang, et al., Codexglue: A machine learning benchmark dataset for code understanding and generation, 2021, arXiv preprint [arXiv:2102.04664](https://arxiv.org/abs/2102.04664).
- [48] D. Wang, P. Cui, W. Zhu, Structural deep network embedding, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1225–1234.
- [49] U. Alon, S. Brody, O. Levy, E. Yahav, Code2seq: Generating sequences from structured representations of code, 2018, arXiv preprint [arXiv:1808.01400](https://arxiv.org/abs/1808.01400).
- [50] 2022 CWE top 25 most dangerous software weaknesses, 2022, [Online], https://cwe.mitre.org/top25/archive/2022/2022_cwe_top25.html.
- [51] C. Tantithamthavorn, S. McIntosh, A.E. Hassan, K. Matsumoto, Automated parameter optimization of classification techniques for defect prediction models, in: *Proceedings of the 38th International Conference on Software Engineering*, 2016, pp. 321–332.
- [52] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, Gnnexplainer: Generating explanations for graph neural networks, *Adv. Neural Inf. Process. Syst.* 32 (2019).
- [53] T.G. Nguyen, T. Le-Cong, H.J. Kang, R. Widayarsi, C. Yang, Z. Zhao, B. Xu, J. Zhou, X. Xia, A.E. Hassan, et al., Multi-granularity detector for vulnerability fixes, *IEEE Trans. Softw. Eng.* (2023).
- [54] T.G. Nguyen, T. Le-Cong, H.J. Kang, X.-B.D. Le, D. Lo, Vulcurator: a vulnerability-fixing commit detector, in: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 1726–1730.
- [55] J. Zhou, M. Pacheco, Z. Wan, X. Xia, D. Lo, Y. Wang, A.E. Hassan, Finding a needle in a haystack: Automated mining of silent vulnerability fixes, in: *2021 36th IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE*, 2021, pp. 705–716.
- [56] C. Thapa, S.I. Jang, M.E. Ahmed, S. Camtepe, J. Pieprzyk, S. Nepal, Transformer-based language models for software vulnerability detection, in: *Proceedings of the 38th Annual Computer Security Applications Conference*, 2022, pp. 481–496.
- [57] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, Graph attention networks, 2017, arXiv preprint [arXiv:1710.10903](https://arxiv.org/abs/1710.10903).